

INTERNAL ENGINEERING REFERENCE

AXpress Knowledge Base

A complete architectural and operational reference for the AXpress (Assured Express) logistics & delivery platform monorepo — backend services, frontend applications, integrations, and infrastructure.

Repository: aexpress (monorepo)

Generated: September 3, 2026

Scope: backend/, frontend/, dispatcher-frontend/, routing_service/, functions/, and supporting docs

Table of Contents

1 Product Overview

2 Tech Stack Summary

3 Monorepo Structure

4 Backend: Django Apps Inventory

5 Backend: Domain Models & Key Endpoints

6 Third-Party Integrations

7 Routing Service (Go)

8 Background Jobs & Celery Schedule

9 Authentication & Authorization

10 Testing Strategy

11 Merchant Portal (Next.js)

12 Dispatcher / Operations Portal

13 Serverless Functions

14 Deprecated & Legacy Artifacts

15 Documentation & Postman Index

16 CI/CD & Deployment

17 Project Tooling & Process

18 Known Issues & Open Risks

1. Product Overview

AXpress is a next-generation logistics and delivery platform for merchants in Lagos, Nigeria. It connects merchants who need deliveries with a managed fleet of riders/dispatchers, offering real-time tracking, automated zone-based dispatch, and AI-assisted operations.

Core Value Propositions

- **Scalability** — built around zone/vertical segmentation and multi-hop relay routing for both local and cross-zone deliveries.
- **Intelligence** — Google Vertex AI / Gemini powers an AI support-chat agent and an experimental AI order-booking flow.
- **Real-time** — Ably pub/sub drives live rider GPS tracking, order-offer broadcast, and dispatcher activity feeds.
- **Financial integration** — merchant/rider wallets, Paystack funding, dedicated virtual bank accounts (CoreBanking/LibertyPay), and prepaid/postpaid subscription billing.

Primary User Roles

Role	Surface	Description
Merchant	Merchant Portal (Next.js)	Creates orders, funds wallet, tracks deliveries, manages staff/subscriptions.
Rider	Rider mobile app (not in this repo — see BACKEND_API_REQUIREMENTS.md)	Accepts order offers, executes pickups/dropoffs, earns wallet payouts, participates in gamification (streaks, challenges, Ride-to-Own financing).
Dispatcher / Ops staff	Dispatcher/Operations Portal (Vite)	Monitors live fleet, assigns riders, manages merchants/zones/pricing, handles messaging.
Zone Captain / Vertical Lead / Admin	OCC & Operations Dashboard APIs	Oversees zone/vertical performance, targets, leaderboards, alerts.
Customer	Receiving end of a delivery	Tracked as a <code>usertype</code> on the shared User model; receives order status via WhatsApp/SMS.

2. Tech Stack Summary

Backend

Django 4.2

Django REST Framework

PostgreSQL

Redis

Celery + Beat

SimpleJWT

Google ADK / Vertex AI
(Gemini)

Ably

boto3 / S3

Firebase Admin (FCM)

Despite the README describing "Django + FastAPI," no active FastAPI application exists in the codebase —

FastAPI/Starlette/uvicorn are transitive dependencies of the Google ADK agent tooling only. All HTTP APIs are served through Django DRF.

Merchant Frontend

Next.js 16 (App Router)

React 19

TypeScript

Tailwind CSS 4

Framer Motion

Ably SDK

React Context for auth state; no Redux/Zustand. Hardcoded production API base URL.

Dispatcher / Ops Portal

Vite 6

React 19

Ably SDK

JavaScript (JSX)

Almost entirely a single ~7,600-line component file; env-driven API base URL with automatic 401 refresh-token retry logic.

Routing Microservice

Go

Gin

OSRM

Docker

Standalone REST wrapper around a self-hosted OSRM instance for road-distance/duration lookups, decoupled from the Django backend.

Infrastructure & Ops

- **CI:** CircleCI — single test-gate job on every PR (no lint/deploy stage in CI).
- **Deployment:** Vercel (frontend apps); backend deployment path not covered by the researched files (see `backend/deploy/`).
- **Serverless:** DigitalOcean Functions (OpenWhisk) for isolated Google Maps distance lookups.

3. Monorepo Structure

Path	Purpose
backend/	Django project <code>ax_merchant_api</code> — all production APIs, models, Celery tasks.
frontend/	Next.js 16 Merchant Portal (active).
dispatcher-frontend/	Vite + React Operations/Dispatcher Portal (active).
dispatcher-portal/	Deprecated — stale build artifacts only, no source remains.
Depreacated (dispatcher-portal) Depreacated/	Deprecated — last snapshot of an abandoned modular TypeScript rewrite of the dispatcher portal.
routing_service/	Standalone Go/Gin microservice wrapping a self-hosted OSRM instance.
functions/	DigitalOcean serverless functions for Google Maps distance calculations.
MerchantPortal.jsx, api.js	Legacy pre-Next.js single-file merchant portal prototype and its API layer (superseded).
*.postman_collection.json	Postman collections documenting merchant, dashboard, and WhatsApp bot API surfaces.
*.md (root)	Standalone architecture/requirements/audit docs — see §15.

4. Backend: Django Apps Inventory

The Django project package is `backend/ax_merchant_api`. All apps below are registered in `INSTALLED_APPS`.

App	Responsibility
authentication	Custom <code>AUTH_USER_MODEL</code> (UUID PK, phone/email login) shared by every role; addresses; email logs.
orders	Order creation (quick-send, multi-drop, bulk), vehicles, pricing overrides, relay legs, Places/geocode proxy, AI booking agent.
wallet	Merchant/rider wallets, Paystack funding, virtual accounts, escrow, vehicle-financing "amortization" wallets.
dispatcher	Largest app: Riders, Merchants, Zones/Verticals, RelayNodes, VehicleAssets, API-key auth, OCC read models/endpoints, snapshotting, ghost-rider detection.
operations	Parallel internal ops-staff dashboard API (<code>/api/operations/v1/</code>) with heavier admin CRUD, gated by <code>IsOperationsAdmin</code> .
oprtn_dashboard	Rule-based Alert/AlertRule engine (ghost rides, speed violations), fuel-bill import, payments/COD dashboards, dashboard caching.
riders	Rider-facing backend: separate rider auth, order offers, earnings, COD, gamification, Ride-to-Own financing, device tokens, demand heatmap.
subscriptions	Prepaid credit-bundle plans and postpaid billing (dedicated riders, invoicing, billing cycles, overage fees).
webhooks	Per-merchant outbound webhook config with reliable delivery outbox/retry pattern.
referrals	Rider referral program and "buddy" commission payouts; syncs LibertyPay user data.
chats	In-app AI support chat — Google ADK multi-agent bot, Ably realtime delivery.
whatsapp_messaging	Outbound WhatsApp message log/sending for order notifications.
bot	WhatsApp merchant self-service bot (signup, orders, quotes, wallet balance) with its own API-key auth.
devs	Internal utility app — custom exception/traceback logger (<code>ErrorLog</code>).

Support directories that are *not* Django apps: `ai_learnings` (reference notes for the ADK integration), `crons`, `data`, `deploy`, `docs`, `scratch`, `templates` (email templates), `test_utils` (shared pytest factories/mocks).

5. Backend: Domain Models & Key Endpoints

authentication — /api/auth/

User (UUID PK, `usertype` : Merchant/Rider/Customer/Dispatcher/ZoneCaptain/VerticalLead, phone as USERNAME_FIELD, OTP + email-verification + password-reset tokens). **Address** (multi-address per user).

```
signup/ · login/ · logout/ · refresh/ · verify-otp/ · resend-otp/ · reset-password/ · me/ · profile/ · addresses/
```

orders — /api/orders/

Order (UUID, order_number, mode: quick/multi/bulk/grouped, status Pending→Assigned→Pickup→Fulfilling→Done, payment_method: wallet/cash/cash_on_pickup/receiver_pays/postpaid/subscription). **OrderLeg** (one relay hop between RelayNode hubs, hub_pin handoff confirmation, per-leg rider payout/bonus). **Vehicle**, **MerchantPriceList/Item**, **GooglePlace** (Places API cost-tracking cache).

```
quick-send/ · multi-drop/ · bulk-import/ · calculate-fare/ · quote/ · pickup/ · start/ · arrived/ · status/ · cancel/<order_number>/ · <order_number>/pay-now/ · track/<order_id>/ · agent/book/ · places/autocomplete · places/details · places/geocode · places/reverse-geocode · smart-parcel/states/
```

wallet — /api/wallet/

Wallet (one-to-one/user, atomic credit()/debit() with row-level locking). **Transaction** (ledger with before/after balances). **VirtualAccount** (dedicated bank account via CoreBanking). **Charge** (deferred order charges). **AmortizationWallet/Transaction/VirtualAccount** (separate ledger for vehicle-financing installments).

```
balance/ · transactions/ · fund/initialize/ · fund/verify/ · webhook/ · virtual-account/ · fund/transfer-claim/ · corebanking-webhook/ · amortization-wallet/ · amortization-transactions/
```

dispatcher — core ops entity + OCC (/api/dispatch/, /api/occ/)

Vertical (org unit, code A/B/C/D + lead), **Zone** (center lat/lng + radius geofence), **RelayNode** (handoff hub), **VehicleAsset/VehicleTracking**, **Rider** (6-digit rider_id, status online/on_delivery/offline, GPS fields, Onro migration fields, soft-delete), **Merchant** (activity_status active/watch/inactive, merchant_type regular/api), **DispatcherProfile** (role: zone_lead/hub_captain/admin), **ServiceAPIKey/MerchantAPIKey**, **RiderDailySnapshot/MerchantDailySnapshot**, **DeliveryRating**, **ZoneTarget**.

```
Dispatch: settings/ · riders/onboarding/ · s3/presigned-url/ · activity/ · ably-token/
```

OCC (ServiceAPIKey auth, occ:read/occ:write scopes): verticals/ · verticals/<id>/zones/ · zones/<id>/dashboard|riders|merchants · riders/<id>/performance · riders/locations/ · merchants/<id>/analytics · leaderboard/zones|verticals · orders/analytics/ · zone-targets/

operations — /api/operations/v1/

Parallel internal-staff dashboard reusing Zone/Rider/Vertical/Merchant data, gated by **IsOperationsAdmin** (superuser or dispatcher admin role). Adds heavier admin CRUD: **admin/zones/**, **admin/hubs/**, **admin/zone-targets/**, **admin/vertical-leads/**, **admin/zone-captains/**, **admin/riders/<id>/hub/**, plus **km-visibility/** and **km-integrity/** fleet-distance auditing.

oprtn_dashboard — /api/ops/

Alert/AlertType/AlertRule/Severity/AlertStatus (ghost rides, speed violations, bike-after-hours), **FuelBill** import. Celery generates alerts every 30 min and refreshes cached dashboard snapshots.

health/ · alerts/ · payments/ · fuel/upload/

riders — /api/riders/

RiderAuth (independent credential store — password/PIN/biometric, lockout after 5 failed attempts), **RiderSession**, **RiderCodRecord**, **RiderEarning**, **RiderDocument**, **OrderOffer**, **RiderDevice**, **AreaDemand**, **RideToOwnConfig/Enrollment**, **RiderMonthlyTarget**, **RiderStreak**, **Challenge/RiderChallengeProgress**, **LeaderboardEntry**.

auth/login · auth/refresh · auth/me · duty/ · orders/offers/ · earnings/ · wallet/info/ · orders-today/ · ride-to-own/ · monthly-target/ · challenges/ · leaderboard/

subscriptions — /api/subscriptions/

SubscriptionPlan (price, free_orders_limit, order_credits, overage_fee%), **MerchantSubscription** (atomic deduct_credit), **SubscriptionUsage/Overage/Invoice**, separate **PostpaidPlan/MerchantPostpaidSubscription/PostpaidInvoice** track, **MerchantDedicatedRider**.

6. Third-Party Integrations

Integration	Used For
Paystack	Card/bank wallet funding — initialize/verify/webhook flow.
CoreBanking (LibertyPay Banking)	Dedicated Wema Bank virtual accounts per merchant.
LibertyPay API	Cross-product referral matching (LibertyPayUser sync).
Ably	Realtime pub/sub: rider location, order-offer push, chat, dispatcher activity feed.
AWS S3 (boto3)	Rider document/photo storage.
AWS Location Service	Configured place index (secondary; not the primary geocoder in use).
Google Maps API / Geoapify / Mapbox	Geocoding & places autocomplete (note: a hardcoded Geoapify fallback key exists in settings).
Google Cloud Vertex AI / Gemini / Google ADK	AI support chat agent (<code>chats/ai_engine.py</code>) — coordinator + order-support + technical-support LlmAgents on <code>gemini-3-flash-preview</code> , with a custom Django-backed ADK memory service.
Firebase Admin (FCM)	Push notifications to rider devices.
Mailgun / MailNow	Transactional email (two provider configs coexist — migration in progress).
respond.io / 360Messenger	WhatsApp outbound messaging providers (also appears to be an in-progress migration).
SmartParcel	External parcel-locker logistics partner integration.
Onro	Legacy dispatch platform being migrated away from (Rider model retains <code>onro_driver_id</code> fields).
GPSWOX	GPS tracking device integration.
Custom Go Routing Service	Distance/duration for relay-leg construction; toggle via <code>ROUTING_PROVIDER=custom google</code> .
Redis	Cache backend, session store, Celery broker/result backend.

Note: No Sentry or equivalent error-tracking service is configured anywhere in the backend. Error logging instead relies on a custom `devs.ErrorLog` model and internal logger utility.

7. Routing Service (Go)

A standalone Go microservice (Gin framework) at `routing_service/`, fully decoupled from the Django codebase — no shared code, communicates only via REST. It wraps a self-hosted **OSRM** (Open Source Routing Machine) instance and exposes:

- `/api/v1/directions` — turn-by-turn route + distance/duration between two points.
- `/api/v1/table` — distance matrix for multiple points.

Runs standalone via Docker (container port 8081, host-mapped 8071). The Django backend calls it via `ROUTING_SERVICE_URL`, primarily in `dispatcher/tasks.py` when building relay-delivery leg routes, with a configurable fallback to Google's Directions API. Effectively a self-hosted cost-saving alternative to Google Directions.

8. Background Jobs & Celery Schedule

Broker/result backend: Redis. Configured in `backend/ax_merchant_api/celery.py`, autodiscovering `tasks.py` per app.

Task	Schedule	Purpose
<code>riders.tasks.publish_random_order_offer</code>	Every minute	Broadcasts order offers to available riders.
<code>webhooks.tasks.webhook_retry_cron</code>	Every 30s	Retries failed outbound webhook deliveries.
<code>dispatcher.tasks.aggregate_daily_rider_snapshots</code>	Daily 00:05	Daily rider performance rollups.
<code>dispatcher.tasks.aggregate_daily_merchant_snapshots</code>	Daily 00:10	Daily merchant performance rollups.
<code>dispatcher.tasks.update_merchant_activity_status</code>	Every 6h	Recomputes merchant active/watch/inactive status.
<code>dispatcher.tasks.flag_ghost_riders</code>	Every 15 min	Detects riders online with no GPS movement/activity.
<code>orders.tasks.process_daily_drip_campaigns</code>	Daily 08:00	Marketing drip emails.
<code>orders.tasks.process_weekly_monday_reports</code>	Mondays 09:00	Weekly merchant reports.
<code>subscriptions.tasks.process_subscription_invoicing</code>	Daily 00:00	Prepaid subscription invoicing.
<code>subscriptions.tasks.process_postpaid_billing_cycles</code>	Daily 00:00	Postpaid billing cycle processing.
<code>oprtn_dashboard.tasks.generate_alerts</code>	Every 30 min	Runs the full alert-rule engine.
<code>oprtn_dashboard.tasks.refresh_dashboard_cache</code>	Every 30 min	Pre-warms payments/COD/fuel dashboard caches.

Other notable triggered (non-scheduled) tasks: order proximity processing, order-charge creation, order-completion handling, transactional email sending, the relay-routing pipeline (leg generation, rider assignment, vertical-lead notification, S3 doc upload), referral commission payouts/LibertyPay sync, async AI chat responses, and WhatsApp message sending.

9. Authentication & Authorization

- **Primary scheme:** JWT via `rest_framework_simplejwt`, with unusually long-lived tokens — both access and refresh tokens default to **90 days**, with rotation and blacklist-after-rotation enabled.
- **Single shared User table** across all roles; role-specific profile rows (`Merchant`, `Rider`, `DispatcherProfile`) hang off it via OneToOne relations, and permission classes check for their presence.
- **Riders have a second, independent credential store** (`riders.RiderAuth`: password/PIN/biometric, own lockout and OTP flow) separate from the main User's auth fields.
- **Role-based permissions:** `IsDispatcher`, `IsZoneLead`, `IsDispatcherAdmin`, `IsMerchant`, `IsOperationsAdmin`.
- **Service-to-service auth:** `ServiceAPIKeyAuthentication` — SHA-256-hashed bearer keys, scope-based (`occ:read` / `occ:write` / `occ:*`), used by OCC endpoints.
- **Bot API-key auth:** shared-secret `X-API-Key` header for the WhatsApp merchant self-service bot, entirely separate from JWT.
- **MerchantAPIKey:** OTP-gated programmatic API access for merchants.
- Django sessions exist (Redis-backed) mainly for the admin site, not API auth.

10. Testing Strategy

- **pytest + pytest-django**, dedicated `settings_test.py` (isolated test DB, MD5 password hasher, `CELERY_TASK_ALWAYS_EAGER`, in-memory cache, mocked external API keys).
- `--reuse-db --no-migrations` for speed; 60s per-test timeout; coverage gate **fail_under = 80%**.
- Markers: `unit`, `integration`, `api`, `celery`, `slow`, `external` (external tests skipped in CI).
- Every app has its own `tests/unit/` and `tests/integration/` structure (standardized per the 2026-08-27 changelog entry).
- Shared fixtures/mocks in `backend/test_utils/` (factory_boy factories, autouse mockers for Celery/external HTTP/signals).
- A number of ad-hoc `verify_*.py` scripts sit at the backend root — manual smoke-test scripts used during feature development, not part of the pytest suite.

11. Merchant Portal (Next.js)

Next.js 16 App Router application at `frontend/`. Routes: landing, login/signup/verify-email/forgot-password/reset-password, dashboard, orders, new-order, wallet, loans, accounting, settings, support, plus stub pages for a future POS terminal (`webpos/`) and website builder (`website/`).

Merchant Capabilities

- Signup/login with OTP verification.
- Create orders — Quick Send, Multi-Drop, Bulk Import (CSV) with pre-booking fare calculation.
- Track, cancel, and pay for orders; rate riders post-delivery.
- Wallet funding (Paystack or virtual bank transfer), balance and transaction history.
- Business loans screen, staff/team management, live AI support chat.
- SmartParcel locker integration and Google Places address autocomplete.

Architecture Notes

- **State:** React Context (`AuthContext`) + local component state — no Redux/Zustand.
- **API layer:** `src/lib/api.ts` — **hardcoded** production API base URL (`https://www.orders.express.net/api`), no automatic 401 refresh-retry logic.
- **Realtime:** Aply used only for the support-chat channel, not for order/location feeds on the merchant side.
- Only one environment variable in active use: `NEXT_PUBLIC_GOOGLE_MAPS_API_KEY`.

12. Dispatcher / Operations Portal

Vite + React SPA at `dispatcher-frontend/` — the internal "back office" for dispatchers/ops staff. Unusually, nearly the entire app lives in one ~7,600-line file, `AX_Dispatch_Portal.jsx`, mounted via a thin `src/main.jsx` entry point; navigation is state-driven rather than URL-routed.

Screens

Login/password-reset, a main shell with live Ably feeds, Dashboard, Orders (assign riders, change status, cancel, event logs), Riders, Vehicles (fleet/asset management), Merchants, Customers, Messaging (Ably live chat), Settings (pricing, dispatch rules, price simulator), Teams.

Architecture Notes

- **Realtime:** subscribes to a `dispatch-feed` channel (order/status events) and a `vehicle-telemetry` channel (live GPS), with a 10-second polling fallback if Ably disconnects; a second independent Ably client is used for messaging.
- **API layer:** env-driven (`VITE_API_BASE_URL`) with a more robust `fetchWithAuth` wrapper — automatic 401 → refresh-token retry with a request queue to avoid duplicate refresh calls.
- Minimal dependency footprint — no UI framework, router, or state library; everything hand-rolled.

13. Serverless Functions

`functions/` deploys to **DigitalOcean Functions** (OpenWhisk-based, via `doctl serverless`) — not Firebase or Vercel. A single package, `routing`, exposes two Python 3.11 actions:

- `calculate_single` — origin/destination → Google Maps Distance Matrix lookup, returns `distance_km/duration_minutes`.
- `calculate_multiple` — same pattern for multiple destinations (multi-drop fare/route calculation).

Both enforce origin-header allow-listing (`ALLOWED_DOMAINS`). Purpose: isolate the Google Maps API key and its access control away from the main Django backend.

14. Deprecated & Legacy Artifacts

Artifact	Status
dispatcher-portal/	Empty shell — only stale <code>.env</code> / <code>dist</code> / <code>node_modules</code> remain, no source. Superseded by <code>dispatcher-frontend</code> .
Depreacated (dispatcher-portal) Depreacated/	Last snapshot of an abandoned modular TypeScript/Vite rewrite of the dispatcher portal. Git history shows an explicit revert commit ("SWITCH BACK TO JSX ON DISPATCHER") back to the current single-file JSX approach.
MerchantPortal.jsx	358KB pre-Next.js single-file merchant portal prototype (global React/ReactDOM, no modules). Superseded by <code>frontend/</code> .
api.js (root)	Matching API layer for the legacy MerchantPortal.jsx prototype, superseded by <code>frontend/src/lib/api.ts</code> .
check.py	Developer scratch file — contains an unrelated pasted Django snippet plus draft <code>AblyRestPublisher</code> utility code.
payload.json	Manual test payload matching the <code>calculate_multiple</code> serverless function's expected request shape.

15. Documentation & Postman Index

Root Markdown Documents

Document	Covers
BACKEND_API_REQUIREMENTS.md	Endpoint spec/checklist for the separate AX Rider mobile app (not in this repo), with criticality ratings per endpoint.
ENDPOINTS_DOCUMENTATION.md	Broadest single API reference — ONRO integration, dashboards, wallet, auth, subscriptions, SmartParcel, dispatcher orders, Places/AWS fallback.
OPERATIONS_DASHBOARD_API_GUIDE.md	Frontend-facing guide to the <code>/api/operations/v1/</code> namespace and role scoping.
OPERATIONS_DASHBOARD_IMPLEMENTATION_PLAN.md	Backend implementation plan establishing <code>/api/operations/v1/</code> as successor to legacy <code>/api/occ/</code> .
OPERATIONS_ENDPOINT_AUDIT.md	Live-data audit of Operations Dashboard endpoints flagging production data-quality bugs (see §18).
OCC_INTEGRATION_AUDIT.md	Earlier audit mapping backend endpoints against Operations Command Center dashboard needs.
ZONE-IMPLENETATION.md	Design doc for the Point+Radius zone model and multi-hop rider relay logic (OSRM-based leg splitting, Ably handoff coordination).
CHANGELOG.md	Ongoing change log — recent entries cover test-suite standardization, Places API cost tracking, fare guardrails, webhook migration fixes.

Postman Collections

Collection	API Surface
AX_Merchant_Dashboard_API_Postman_Collection.json	Authentication, Profile & Addresses, Orders, Wallet (broadest merchant-facing surface).
AX_Merchant_Portal.postman_collection.json	Authentication, Orders, Wallet (earlier/narrower variant).
AX_WhatsApp_Bot_API_Postman_Collection.json	Authentication, Dashboard, Orders, Wallet — for the WhatsApp bot ordering channel.

16. CI/CD & Deployment

- **CircleCI:** single `test` job (ubuntu-2404), decodes a base64 env secret and runs `./test.sh` on every PR (10-min timeout). No lint or deploy stage in CI.
- **Vercel:** root `vercel.json` sets security headers plus an SPA fallback rewrite (`/(.*) → /index.html`) — a Vite-style pattern, suggesting it targets one of the dispatcher portal builds rather than the Next.js app. `frontend/vercel.json` sets only security headers (appropriate for Next.js's native routing).

17. Project Tooling & Process

- **Agent skills** (`.agents/skills/` , sourced from `Ayobami6/skills`): `clean-code` (SOLID/TDD/refactoring standards), `mailnow-integration` (MailNow transactional email SDK/MCP setup), `pr-reviewer` (GitHub MCP-based PR review checklist).
- **Project tracking**: ClickUp, mapped 1:1 to product surfaces — `MERCHANT_WEB` , `MERCHANT_APP` , `RIDER_APP` , `OPERATION_DASHBOARD` , `DISPATCHER` lists (`.claude/clickup.env` template).
- **Engineering directives** (`GEMINI.md`): hexagonal/DDD architecture principles, immutable double-entry ledger rules for financial logic, idempotency and distributed-locking requirements for concurrency, and strict per-language conventions (DRF `APIView` -only — `generics/ViewSets` disallowed; Django Ninja preferred for new APIs; strict TypeScript; Conventional Commits + mandatory `pr-reviewer` self-review before human merge).

18. Known Issues & Open Risks

Compiled primarily from `OPERATIONS_ENDPOINT_AUDIT.md` (2026-06-18 live-data audit) and cross-referenced against the codebase:

Merchant activity misclassification — merchants are frequently misclassified "inactive" because `last_order_date` is not updated by the real order-completion flow.

KM-integrity false positives — fleet distance-integrity flags are reported as 100% false positives because a required cron/management command is never actually run in production.

Revenue targets always zero — no `ZoneTarget` rows exist in the live database, so target-based dashboard metrics report 0 across the board.

Hardcoded fallback API key — a Geoapify API key fallback is hardcoded directly in `settings.py` rather than sourced purely from environment configuration.

Hardcoded superuser credentials — `BACKEND_API_REQUIREMENTS.md` contains a hardcoded superuser credential block; this document should be treated as sensitive and access-restricted.

Provider migrations in flight — two coexisting configs each for email (Mailgun/MailNow) and WhatsApp (respond.io/360Messenger) suggest incomplete provider migrations; verify which is authoritative before building on either.

Long-lived JWTs — access and refresh tokens both default to a 90-day lifetime, which is unusually long and worth revisiting from a security-posture standpoint.

No centralized error tracking — the absence of Sentry (or equivalent) means production errors rely entirely on the custom `devs.ErrorLog` model and manual log inspection.

This knowledge base was compiled by static analysis of the repository on September 3, 2026, and reflects the code and documentation as of that date. Treat any operational claims (integration status, live-data findings) as a snapshot, not a guarantee of current production behavior.